

**CENTRALE
LYON**

MOD : DATA SCIENCE
RECHERCHE D'INFORMATION DANS LA LITTÉRATURE
SCIENTIFIQUE

Rapport BE2 : Moteur de Recherche

Élèves :

Adrien LOUIS
Quentin POTIRON
Timothée ROBERT

Enseignants :

Erwan VERSMEE

9 janvier 2026

Table des matières

1	Introduction et définition du problème	2
1.1	Contexte	2
1.2	Objectifs du Projet	2
1.3	Présentation des données	2
2	Exploration et analyse des données	3
2.1	Quelques statistiques du corpus	3
3	Approche creuse	3
3.1	Construction du dictionnaire	4
3.2	Exploitation de l'approche creuse	5
4	Approche dense	6
5	Graphes de liaisons	7
5.1	Étude du graphe de citations	7
5.1.1	Statistiques du graphe	8
5.1.2	Mesure de la centralité	8
5.2	Étude du graphe des auteurs	9
5.2.1	Statistiques du graphe	9
5.2.2	Centralité du graphe	9
5.3	Implémentation des graphes à l'approche dense	10
5.4	Post-traitements avec citations et références	10
6	Analyse des résultats obtenus et perspectives d'améliorations	11
7	Conclusion	12
8	Bibliographie	13
9	Annexe	14
9.1	Indicateurs pour moteur de recherche	14
9.2	Importance du graphe citation	14

1 Introduction et définition du problème

1.1 Contexte

Ce projet s'inscrit dans le cadre du module "Introduction à la Data Science". Face à l'augmentation exponentielle de la quantité de données publiées sur internet, l'optimisation de l'analyse de l'ensemble de ces données est devenu un enjeu majeur pour les entreprises du secteur.

Ce rapport décrit donc notre travail réalisé sur l'exploitation des données d'un corpus pour construire un moteur de recherche le plus performant possible.

1.2 Objectifs du Projet

L'objectif principal est de développer un algorithme capable de mesurer la similitude entre une publication cible (la requête q) et un ensemble de candidats C . Plus précisément, pour une requête donnée, nous devons :

- Identifier les 5 articles effectivement liés la requête (exemples positifs C^+).
- Les distinguer d'un ensemble d'environ 25 articles choisis aléatoirement (exemples négatifs C^-).

L'objectif principal de cette étude est donc d'être capable d'associer un score de similitude entre deux documents suffisamment robuste et discriminant afin d'identifier les 5 documents associés à notre requête q et d'écarter les autres documents.

Ainsi, nous aurons construit un moteur de recherche à une petite échelle.

1.3 Présentation des données

Pour mener à bien cette tâche, nous disposons de trois fichiers principaux fournies sous formats .jsonl et .tsv :

- corpus.jsonl : La base de référence contenant les métadonnées de plus de 25 000 articles (titres, résumés, auteurs, années, références).
- queries.jsonl : L'ensemble des documents servant de point de départ à nos recherches, c'est à dire les requêtes q , au nombre de 1000.
- valid.tsv : Le document nous permettant d'entraîner notre modèle, il contient les requêtes ainsi que les 5 documents positifs à identifier et les autres documents négatifs à écarter.
- test_final.tsv : Le document de test sur lequel on utilise notre modèle et envoyer le résultat à la compétition Kaggle.

L'enjeu sera d'extraire et de transformer ces informations textuelles et structurales (auteurs, citations) en vecteurs exploitables par nos modèles.

2 Exploration et analyse des données

Une fois que nous avons chargé les données sur notre machine, nous pouvons commencer par faire une analyse succincte des données du corpus. Cette étape préalable à l'étape de codage est nécessaire pour appréhender le jeu de données et comprendre ses subtilités s'il y en a, ou pour si possible, procéder à un pré-traitement.

2.1 Quelques statistiques du corpus

Parmi les documents à notre disposition, nous avons exactement 26657 documents dans le corpus, ainsi que 1000 queries.

En regardant tout d'abord les caractéristiques décrivant chaque document, on remarque une première différence notable :

Les documents du corpus disposent d'un *Titre* ET d'un *Texte* résumant le contenu de ce document, alors que les queries ne disposent que d'un *Titre* qui est d'ailleurs sous le nom de "Text", il faut donc être vigilant pour ne pas faire de confusion.

Nous avons également essayé de survoler quelques documents du corpus pour éventuellement identifier des éléments particuliers auquel on pourrait apporter un soin particulier lors de l'étape de pré-traitement mais nous n'avons pas relevé quoi que ce soit de pertinent.

Nous avons constaté plusieurs documents faisant mention de Copyright et avons initialement envisagé d'éliminer ces parties de texte, mais nous avons finalement décidé de les conserver de par le fait que la mention d'un Copyright dans deux documents distincts apporte une réelle similitude entre ces-derniers.

Nous avons aussi relevé que dans le corpus, les documents contiennent assez régulièrement un nombre assez important de références ou de documents cités, et c'est une chose que l'on exploitera par la suite via un graphe des citations.

3 Approche creuse

La première approche utilisée pour construire un moteur de recherche est basée sur une approche creuse : il s'agit de représenter chaque document (et chaque requête) par un vecteur de caractéristiques où seuls quelques termes sont présents, les autres ayant une valeur nulle.

Pour cela, il faut :

- construire un dictionnaire global ;
- associer à chaque document un vecteur associé au dictionnaire global suivant différentes méthodes :
 - TF (Term Frequency) ;
 - TF-IDF (Term Frequency-Inverse Document Frequency) ;
- récupérer, pour chaque requête, son vecteur dans le dictionnaire global des documents ;

- rechercher les documents avec un vecteur similaire grâce au cosinus entre les vecteurs.

3.1 Construction du dictionnaire

La construction du dictionnaire se fait grâce au *CountVectorizer* de Scikit-learn et sa méthode `fit_transform`. Il faut le configurer comme suivant :

- ignorer les mots vides (stop words) anglais ;
- utiliser seulement les versions des mots en minuscule pour construire le dictionnaire ;
- prendre en compte que les mots d'une longueur supérieure à 2 lettres.

Après application de cette fonction sur une liste contenant les données des documents au format "titre texte", on obtient alors un dictionnaire de 76703 mots. Voici la distribution des mots à travers l'ensemble de notre corpus :

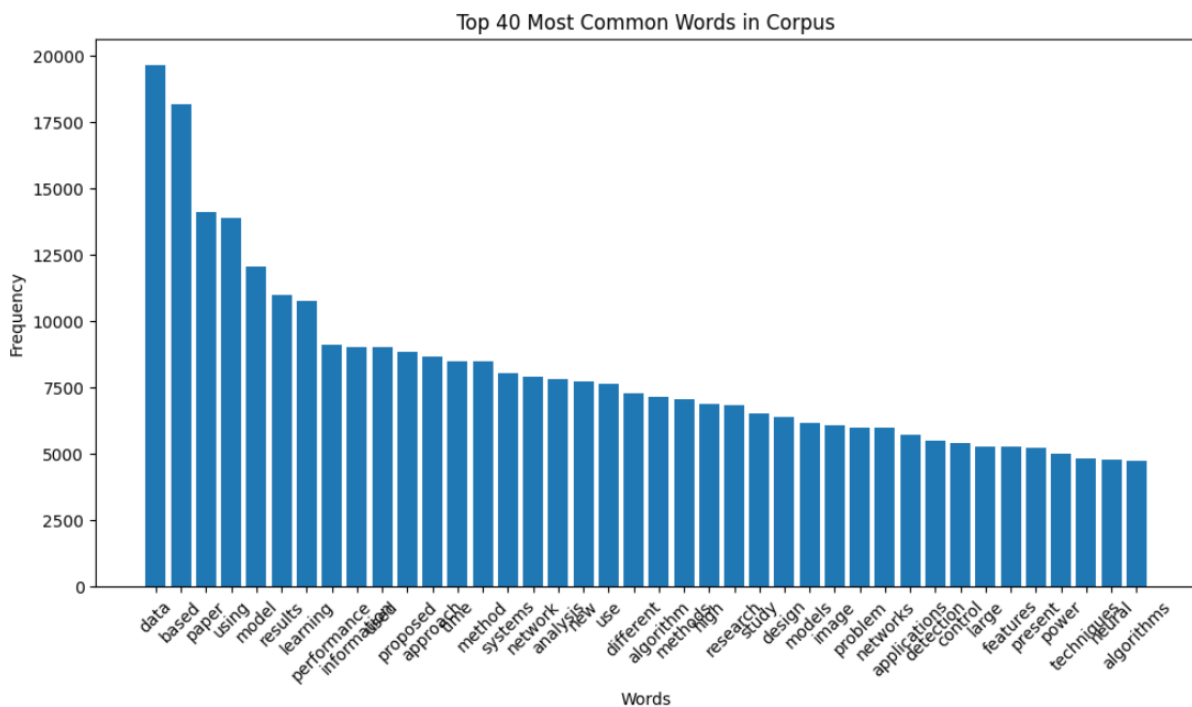


FIGURE 1 – Distribution des mots dans le corpus

La figure 1 montre qu'il y a donc une minorité de mots très présents. De plus, bien que cela ne soit pas représenté sur ce graphe, il y a également une grande majorité de mots qui n'apparaissent qu'une fois, dont des apparitions telles que :

- 000buildings
- cmifornia
- kcxc

Outre leur apparence particulière, le fait qu'ils soient uniques nous indique qu'il y a peu de chance qu'ils soient déterminants dans la recherche des documents car trop spé-

cifiques. Nous avons donc choisi pour notre étape de pré-traitement, de nous séparer de l'ensemble des mots n'apparaissant qu'une seule et unique fois, afin de minimiser le bruit, et nous avons également supprimé les 7 mots les plus présents.

De cette manière, la similitude entre deux documents sera moins perturbée par du bruit, et moins faussée par des mots trop récurrents qui rendrait beaucoup trop de documents similaires entre eux. De plus, le vocabulaire est ainsi réduit de plus de 33 000 mots, ce qui améliore donc grandement la rapidité d'exécution de notre code.

3.2 Exploitation de l'approche creuse

Dans notre code, nous avons choisi de concaténer l'ensemble des données composant un document ou une query, nous laissant avec des "Sacs de mots". Au cours de cette étape nous prenons également le soin de nous laisser la possibilité d'associer un poids à chaque paramètre. Ainsi, si on associe un poids de 10 à la section titre, celui-ci sera alors répéter 10 fois dans notre sacs de mots. Cette approche n'est pas idéal au niveau de la complexité de notre algorithme, mais cela ne génère pas une très grande augmentation du temps de calcul.

Une fois nos données sous la forme de sacs de mots, on applique alors simplement une comparaison par cosinus "cosine similarity" qui va alors comparer deux sacs de mots et nous renvoyer un score entre 0 et 1 pour représenter la similitude entre deux documents.

Nous avons d'abord procéder à différents essais avec des poids différents sur nos données non filtrées, donc contenant tous les mots, dont voici un résumé sur la table 1 ci-dessous :

TABLE 1 – Synthèse des performances selon les pondérations sur les données non filtrées

Configuration	Tit/Txt	Aut	Réf	Cit	Année	Rappel	Préc.	F1-Score
Baseline (1 partout)	1	1	1	1	1	0,625	0,616	0,620
Boost Auteurs	1	10	1	1	1	0,674	0,664	0,669
Boost Texte	3	1	1	1	1	0,619	0,610	0,615
Boost Références	1	1	10	1	1	0,694	0,684	0,689
Boost Citations	1	1	1	10	1	0,554	0,546	0,550
Boost Année	1	1	1	1	10	0,460	0,453	0,456
Boost Auteurs+Réf	1	10	10	1	1	0,699	0,689	0,694

On remarque donc après quelques essais triviaux que nous parvenons à dégager une combinaison qui nous permet d'obtenir un F1 score d'environ 0.7 ce qui n'est pas si mauvais pour une approche creuse de ce type.

Nous allons maintenant essayer d'utiliser la même combinaison avec nos données filtrées. Les résultats sont montrés sur la table 2 juste ci-dessous :

TABLE 2 – Synthèse des performances selon les pondérations sur les données filtrées

Configuration	Tit/Txt	Aut	Réf	Année	Rappel	Préc.	F1-Score
Baseline (1 partout)	1	1	1	1	0,639	0,629	0,634
Boost Auteurs+Réf	1	10	10	1	0,676	0,666	0,671

On remarque ici que pour le cas simple où les poids sont équilibrés, le F1-score est meilleur avec les données filtrées. En revanche, pour notre composition "optimale" identifiée précédemment, on obtient cette fois-ci des résultats moins performants.

Cela met en avant le fait qu'une telle approche est très sensible à la répartition statistique des mots et qu'une étape de pré-filtrage, aussi légère soit-elle, peut venir perturber nos calculs de similitudes et affaiblir la précision de notre modèle.

Il serait intéressant de mener une étude bien plus poussée sur la sélection des poids pour chaque catégorie en testant un grand nombre de combinaisons. Cependant, compte tenu de la rapidité d'exécution de notre algorithme, nous préférons nous orienter vers d'autres méthodes nous permettant de nous affranchir de cette approche en "sacs de mots" qui est pour le moins contraignante.

4 Approche dense

L'approche creuse a de nombreux avantages et de nombreux inconvénients. Aujourd'hui, une manière d'améliorer les représentations est de passer par des représentations denses. Pour cela, nous n'allons pas programmer nous-même une nouvelle représentation dense mais en utiliser une déjà existante.

Pour cela, nous avons utilisé la librairie sentence-transformer [5] qui propose de nombreux modèles prêt à être utilisés. De plus, nous nous sommes aidés d'un benchmark [3] de huggingface afin de trouver un modèle performant utilisable avec les ressources (processeur, RAM, GPU) à notre disposition.

Nous avons utilisé plusieurs modèles : e5-small-v2, all-MiniLM-L6-v2, GIST-small-Embedding-v0 et multilingual-e5-small. Ils ont tous été pris d'après le benchmark. Nous avons fait le choix de prendre des petits modèles afin qu'ils puissent tourner en un temps raisonnable sur nos ordinateurs. Vous trouverez ci-dessous sur la table 3 un comparatif des résultats que nous avons eu selon les modèles et sur la table 4, il y a un comparatif des modèles d'après le benchmark avec le temps de calcul sur un même ordinateur à notre disposition.

	all-MiniLM-L6-v2	e5-small-v2	GIST-small-Embedding-v0	multilingual-e5-small
Precision	0.8031	0.6980	0.8123	0.6826
Rappel	0.8154	0.7088	0.8240	0.6934
F1 Score	0.8092	0.7034	0.8181	0.6879
AUC	0.9556	0.9076	0.9618	0.9042
MRR	0.9743	0.9542	0.9811	0.9496
NDCG	0.8534	0.7642	0.8613	0.7499
MAP	0.7871	0.6693	0.7956	0.6490

TABLE 3 – Comparaison des résultats des modèles

En plus des indicateurs habituels nous avons rajouté plusieurs indicateurs que nous avons trouvé utile dans le cadre de notre moteur de recherche. Vous trouverez leur descriptions dans l'annexe 9.1.

	all-MiniLM-L6-v2	e5-small-v2	GIST-small-Embedding-v0	multilingual-e5-small
Zero-shot	100%	100%	96%	99%
Memory Usage (MB)	87	127	127	449
Embedding Dimensions	384	384	384	384
Rank	121	88	80	44
Computing time (min)	25	40	80	45

TABLE 4 – Comparaison des modèles

On peut s'apercevoir que ce n'est pas parce qu'un modèle est plus petit qu'il est forcément plus mauvais. De plus, si on regarde le classement des résultats et le classement que fait hugging face sur les modèles, nous nous apercevons que ce n'est pas le même. Ainsi, le choix du modèle dépend des capacités de calcul que nous avons et des résultats que nous avons avec les données réelles plutôt que les résultats d'un benchmark.

5 Graphes de liaisons

Nous avons un très grand nombre d'études qui, chacune, ont plusieurs citations et plusieurs auteurs. Nous nous sommes alors intéressé à ces deux aspects pour construire des graphes de liaisons entre les différents documents de notre corpus. Comme un même auteur va écrire principalement des papiers sur des sujets similaires ou que les citations d'une étude porte sur des études du même thème, nous pensons que cela nous permettra d'affiner un modèle déjà existant.

5.1 Étude du graphe de citations

Nous avons commencé par étudier le graphe des citations qui nous paraît plus prometteur que celui des auteurs. En effet, les documents les plus cités sont généralement considérés comme des documents fiables et importants. Pour un moteur de recherche, il est donc intéressant de mettre en avant des documents faisant consensus et étant largement adoptés au sein de la communauté scientifique que des documents avec peu de notoriété.

On remarquera que cette approche tant à léser les études sur un sujet très précis qui seront donc peu cités mais qui pourtant peuvent être des études qui correspondent bien à la recherche de l'utilisateur.

5.1.1 Statistiques du graphe

Pour mettre en place cette approche, nous nous sommes alors intéressés au graphe des citations entre les différents documents. Il est à remarquer qu'un grand nombre des citations vont vers des documents qui ne sont pas dans la liste de documents que nous avons, c'est donc un graphe incomplet puisque nos données le sont. Nous avons alors les informations suivantes sur le graphe :

- Nombre de noeuds : 25657
- Nombre d'arêtes : 54004
- Densité du graphe : $4,2 \cdot 10^{-5}$
- Degrés entrants (In-degree)
 - Moyenne : 2.1049
 - Variance : 10.3450
 - Écart-type : 3.2164
 - Min : 0, Max : 83
- Degrés sortants (Out-degree)
 - Moyenne : 2.1049
 - Variance : 138.6369
 - Écart-type : 11.7744
 - Min : 0, Max : 758

Le graphe est très grand empêchant une représentation simple de celui-ci avec un grand nombre de noeuds mais un petit nombre d'arêtes en comparaison ce qui fait une densité assez basse. Il y a un grand nombre de documents ne citant ni n'étant cité par aucun autre document de la liste que nous avons.

De plus, nous pouvons voir qu'il y a une très grande différence au niveau de la variance et de l'écart-type entre les degrés entrants et les degrés sortants. Cela s'explique qu'une étude va rarement cité plus d'une centaine d'étude, ici le maximum de citation au sein du corpus que nous avons est 83, alors qu'une même étude peut être citée plus centaines de fois voire milliers.

Cela veut donc dire, sans trop de surprise, qu'il y a quelques études très connues, très citées et grand nombre d'études très peu utilisées par d'autres études.

5.1.2 Mesure de la centralité

Pour étudier notre graphe, nous nous sommes intéressés aux mesures de la centralité de celui-ci. Le but de cette étude est de mettre en valeur les oeuvres "centrales" du graphe. L'idée générale d'un grand nombre de types de mesures de la centralité d'un graphe est d'ajouter un poids élevé aux éléments proche d'un grand nombre d'autres éléments.

Dans notre cadre, cela correspondrait donc à mettre en valeur les éléments souvent cités ou les éléments faisant un pont entre plusieurs disciplines car ceux-ci relieraient deux parties du graphe et seront donc plus proche que les études très pointues qui seront comme des feuille au bout d'une branche.

Pour mettre en place cela, il existe donc un grand nombre de modèles différents : pageRank, eigenvector centrality, closeness, clustering coefficient, etc... Nous en avons implémenté plusieurs et nous avons regardé les différences. En comparant juste les 15 documents les mieux notés de pageRank et eigenvector par exemple, il n'y avait qu'un seul élément en commun. Ainsi, le choix du modèle a grand impact sur les résultats du programme et peut donc grandement influencer le moteur de recherche.

5.2 Étude du graphe des auteurs

Une fois notre étude du graphe des citations menés, nous avons fait celui concernant les auteurs. Deux études sont donc liés entre elles si elles ont au moins un auteur en commun. Ici, nous avons donc un graphe non orienté. Nous avons décidé de faire cette étude pour pouvoir comparer avec le graphe des citations, nous étions partis du principe que celui-ci serait certainement moins concluants car le graphe est limité par le petit nombre d'auteurs d'une même étude et le fait que la base de données que nous avons est incomplète.

5.2.1 Statistiques du graphe

Voici les statistiques du graphe :

- Nombre de noeuds : 25657
- Nombre d'arêtes : 78406
- Densité du graphe : $1,19.10^{-4}$
- Degrés du graphe
 - Moyenne : 3.0559
 - Variance : 36.3287
 - Écart-type : 6.0273
 - Min : 0, Max : 89

Avec ces statistiques, on s'aperçoit qu'il y a en fait plus de liaisons dans ce graphe que dans le graphe des citations. De plus, il y a une variance entre le degré entrant et le degré sortant du graphe de citations. C'est donc un deuxième graphe qui se révèle très intéressant à implémenter dans notre moteur de recherche.

5.2.2 Centralité du graphe

Comme pour le graphe précédent, il n'est pas possible de l'afficher facilement dû à un trop grande nombre de noeuds et d'arêtes. De plus, nous avons le même constat que pour le graphe de citations, il n'y a quasiment aucune étude qui est un noeud important pour plusieurs modèles de centralité.

Excepté pour pageRank, nous remarquons que quelque soit la centralité étudiée, les 10 éléments avec les plus fortes valeurs ont des valeurs très similaires. Cela est dû qu'à

partir du moment qu'un auteur est très prolifique, toutes ces oeuvres sont alors liées entre elles. Elles sont donc toutes autant mise en avant.

5.3 Implémentation des graphes à l'approche dense

Nous avons décidé dans un premier temps de juste de modifier l'embedding de l'approche dense suivant le nombre de plus proche voisin sur l'un des deux graphes. Cela permet donc de mettre en avant les oeuvres citées entre elles ou avec des mêmes auteurs. Nous avons décidé de faire les calculs en ne gardant que le meilleur modèle dense soit `GIST-small-Embedding-v0`.

	Sans	Citations	Auteurs	Citations+Auteurs
Precision	0.8123	0.8540	0.7960	0.8497
Rappel	0.8240	0.8670	0.8076	0.8627
F1 Score	0.8181	0.8604	0.8018	0.8562
AUC	0.9618	0.9769	0.9563	0.9751
MRR	0.9811	0.9808	0.9768	0.9801
NDCG	0.8613	0.8939	0.8491	0.8907
MAP	0.7956	0.8422	0.7784	0.8386

TABLE 5 – Comparaison des résultats en rajoutant les informations sur les graphiques

Nous pouvons facilement voir que l'utilisation du graphe des citations permet une amélioration de notre modèle dense. Toutefois, l'utilisation du graphe des auteurs vient baisser les résultats. Nous allons donc abandonner l'étude du graphe des auteurs pour la suite de notre étude et nous concentrer sur le graphe des citations qui semblent plus prometteur. De plus, afin de compléter cet aspect, nous avons étudié l'influence d'un hyperparamètre de ce modèle. Vous retrouverez l'étude et les résultats dans l'annexe 9.2.

5.4 Post-traitements avec citations et références

Enfin, nous avons une dernière idée en tête afin d'améliorer notre moteur de recherche, l'utilisation des champs `cited_by` et `references` des requêtes. Effectivement, une fois que nous avons les résultats de notre moteur de recherche en regardant les titres, les textes avec l'ajout de poids par le graphe des citations, nous faisons un post-traitement à ces résultats suivant les critères suivants :

- Si la requête a dans ces citations ou références le document, alors on lui augmente le score.
- Si la requête a dans ces citations ou références une citation du document, alors on lui augmente le score.
- Dans le cas contraire, son score est légèrement diminué.

La mise en place de ce post-traitement nous a permis d'améliorer encore un peu nos métriques. Toutefois, celui-ci rend les calculs des indicateurs faux avec des valeurs supérieures à 1 sur des indicateurs limités normalement entre 0 et 1.

Pour autant, les résultats donnés par notre moteur de recherche sont meilleurs. Cela nous a permis d'atteindre un score de **0.9911** sur la compétition Kaggle.

6 Analyse des résultats obtenus et perspectives d'améliorations

L'analyse de nos résultats montre que l'approche dense surpasse significativement l'approche creuse, notamment parce que les requêtes sont constituées uniquement de titres. Là où le TF-IDF échoue si les mots ne correspondent pas exactement, les embeddings capturent la sémantique ce qui nous permet de capturer la similarité entre les thèmes de plusieurs documents.

De plus, la prise en compte des citations que ce soit en pré-traitement pour le mettre lors de l'embedding ou en post-traitement pour modifier les résultats des programmes, est une information très importante qui nous permet d'aller encore plus loin de la justesse de nos résultats. Au contraire, la prise en compte des auteurs ne donne pas de résultats probants.

Par ailleurs, pour approfondir notre étude nous aurions pu également explorer d'autres pistes :

- Une approche mixte (creuse et dense) qui nous permettrait de pré-filtrer via l'approche creuse pour isoler les candidats potentiels puis d'effectuer un deuxième classement sur ces données filtrées via un cross encoder (long à exécuter d'où la pré filtration) qui serait alors capable de comparer chaque mot de la query avec ceux des documents en simultané.
- BERTopic pour combiner la recherche dense et la recherche de thèmes en utilisant des embeddings ainsi qu'au TFxIDF. Utile pour conserver une interprétabilité des résultats
- Une approche neuronale comme Graph Neural Networks qui nous aurait permis d'approfondir notre modèle de graphe de citations en allant au delà des mesures de centralité classique et fusionner les données textuelles.

7 Conclusion

Pour conclure, ce projet nous a permis de suivre le processus complet de la construction d'un moteur de recherche, depuis l'analyse exploratoire des données jusqu'à l'implémentation de modèle pour traiter et comparer des données textuelles.

Notre étude nous a permis de mettre en avant plusieurs points :

- L'approche creuse fournit une base solide mais est très limitée de par le fait que celle-ci n'est capable de prendre en compte que des similitudes entre des mots exacts.
- L'approche dense avec des Transformers a significativement amélioré nos résultats en nous permettant de travailler sur les thèmes des documents et non pas sur une correspondance mot à mot.
- L'approche par un graphe des citations nous a permis d'aborder le problème sous un autre angle en exploitant la structure relationnel du corpus, et combiné à une approche dense, cette approche hybride nous a permis d'obtenir les meilleurs résultats de notre étude

8 Bibliographie

Références

- [1] *Mean Average Precision (MAP) in ranking and recommendations*. URL : <https://www.evidentlyai.com/ranking-metrics/mean-average-precision-map>. (Date de consultation : 07/01/2026).
- [2] *Mean Reciprocal Rank (MRR) explained*. URL : <https://www.evidentlyai.com/ranking-metrics/mean-reciprocal-rank-mrr>. (Date de consultation : 07/01/2026).
- [3] *MTEB Leaderboard*. URL : <https://huggingface.co/spaces/mteb/leaderboard>. (Date de consultation : 01/01/2026).
- [4] *Normalized Discounted Cumulative Gain (NDCG) explained*. URL : <https://www.evidentlyai.com/ranking-metrics/ndcg-metric>. (Date de consultation : 07/01/2026).
- [5] *SentenceTransformers Documentation*. URL : <https://www.sbert.net/>. (Date de consultation : 01/01/2026).

9 Annexe

9.1 Indicateurs pour moteur de recherche

Le premier indicateur est le **Mean Reciprocal Rank (MRR)** [2], il nous permet de mieux cerner la position moyenne du premier élément intéressant que nous sort notre moteur de recherche. Ainsi, plus la valeur de cet indicateur est proche de 1, plus cela veut dire que le premier élément de notre moteur de recherche fait effectivement parti de la liste des 5 éléments intéressants de notre requête. Au contraire, si l'indicateur est proche de 0, il n'y a aucun élément intéressant donné par notre moteur de recherche. Cet indicateur se calcule avec la formule suivante :

$$MRR = \frac{1}{U} \sum_{i=1}^U \frac{1}{rank_i} \text{ où } U \text{ est le nombre de requêtes} \quad (1)$$

Ensuite, nous avons décidé d'ajouter un indicateur qui prend en compte l'ordre des éléments renvoyé par notre moteur de recherche. Après quelques recherches, nous avons trouvés deux indicateurs correspondant à notre besoins : **Normalized Discounted Cumulative Gain (NDCG)** [4] et **Mean Average Precision (MAP)** [4][1]. Ils prennent en compte tous deux la position des réponses mais MAP est bien plus sensible aux premières positions tandis que NDCG prend plus en compte l'ordre global comme il utilise une échelle logarithmique. Comme la plupart des indicateurs, ces deux indicateurs sont compris entre 0 et 1 où 0 signifie un très mauvais résultat et 1 signifie d'avoir toutes les bonnes réponses dans le bon ordre. Les équations pour calculer ces deux indicateurs sont ci-dessous :

$$DCG@K = \sum_{k=1}^K \frac{rel_k}{\log_2(k+1)} \text{ et } NDCG@K = \frac{DCG@K}{IDCG@K} \text{ où } IDCG@K \text{ est la valeur idéale} \quad (2)$$

$$AP@K = \frac{1}{N} \sum_{k=1}^K Precision(k) \times rel_k \text{ et } MAP@K = \frac{1}{U} \sum_{u=1}^U AP@K_u \quad (3)$$

Pour les deux équations, on a $rel_k = 1$ si l'item est à la position, sinon $rel_k = 0$.

9.2 Importance du graphe citation

Lors de l'ajout des données du graphe des citations sur l'embedding avec la méthode dense, nous pouvons faire varier l'importance entre les valeurs de l'embedding original et les valeurs liés à notre graphe. Par défaut, nous avons mis une valeur de 0.3, ce qui signifie que les valeurs de l'embedding original sont multiplié par 0.7 tandis que l'on ajoute 30% des valeurs de l'embedding dû au graphe. En testant différentes valeurs, on a alors les résultats suivants :

	0.2	0.3	0.4	0.5
Precision	0.8477	0.8540	0.8531	0.8506
Rappel	0.8604	0.8670	0.8660	0.8634
F1 Score	0.9540	0.8604	0.8595	0.8569
AUC	0.9758	0.9769	0.9769	0.9761

TABLE 6 – Comparaison des résultats suivant l'importance de l'aspect graphe

La valeur de 0.3 était donc la meilleure. En affinant encore plus, on trouve la valeur de 0.34.